

Trabalho - Design de software

SIMONE DOMINICO

September 2025

1 Descrição

O “Seu Cantinho” é uma rede de lojas de aluguel de espaços para festas, como salões, chácaras e quadras esportivas. A proprietária, Dona Maria, iniciou seu negócio utilizando um sistema simples desenvolvido de forma emergencial para atender às demandas iniciais. Com o crescimento da empresa e a expansão para três estados diferentes, esse sistema tornou-se insuficiente, principalmente por não garantir a consistência das reservas entre filiais e por apresentar limitações de usabilidade e confiabilidade. Assim, é necessário o desenvolvimento de um novo sistema que ofereça suporte às seguintes funcionalidades essenciais:

- (a) Cadastro dos espaços disponíveis para aluguel, com informações detalhadas como fotos, capacidade e preço.
- (b) Gerenciamento das reservas futuras, contemplando data, cliente, espaço alugado e valor pago.
- (c) Controle de pagamentos, indicando se a reserva possui apenas sinal pago ou se já está quitada.

A ausência de uma solução robusta tem gerado problemas recorrentes, tais como: *double-booking* (o mesmo espaço reservado simultaneamente por diferentes clientes), perda de informações financeiras e aumento do tempo de operação. Além disso, a baixa familiaridade de Dona Maria com tecnologia exige uma aplicação simples, intuitiva e tolerante a falhas.

É necessário propor uma solução para o “Seu Cantinho”. O projeto deve englobar:

- **Modelagem UML:** elaboração de diagramas de classes e componentes.
- **Estilo Arquitetural:** escolha fundamentada, com justificativa baseada em requisitos de qualidade como simplicidade, escalabilidade, confiabilidade e facilidade de manutenção.
- **Implementação:** desenvolvimento de um protótipo funcional, incluindo operações CRUD (CREATE, READ, UPDATE e DELETE para usuários, espaços, reservas e pagamentos).

- **Documentação:** relatório descrevendo a arquitetura proposta, o mapeamento UML $\rightarrow$ código, as decisões de design tomadas, os trade-offs identificados e as instruções de execução do sistema.

Independentemente do estilo escolhido, o sistema deve:

1. Expor suas funcionalidades por meio de **APIs REST**, documentadas em formato OpenAPI/Swagger.
2. Ser executável por meio de **Docker Compose**, permitindo o início de toda a implementação com um único comando.
3. Ser disponibilizado em um **repositório Git**, contendo código-fonte, diagramas UML, documentação e instruções de execução.

1.1 Instruções de Desenvolvimento

Para padronizar o desenvolvimento do trabalho e facilitar a avaliação, os grupos deverão seguir as instruções abaixo:

1. **Estilo arquitetural de comunicação obrigatório: REST.** Todas as operações do sistema (cadastro de espaços, reservas, pagamentos e usuários) devem ser expostas como serviços REST, utilizando os métodos necessários. A documentação das rotas deve ser disponibilizada no formato OpenAPI/Swagger ou equivalente.
2. **Execução via Docker.** O sistema deve ser empacotado utilizando **Docker**, de forma que a aplicação e suas dependências (ex.: banco de dados, front-end, back-end) possam ser executadas com um único comando:

```
docker compose up
```

3. **Docker versão 28.4.0** (ou superior).
4. **Disponibilização no Git.** O código-fonte e a documentação devem ser armazenados em um repositório Git público (GitHub, GitLab ou equivalente). O repositório deve conter:
 - Diretório `/src` com o código da aplicação.
 - Diretório `/uml` com os diagramas UML (fonte + imagens).
 - Arquivo `docker-compose.yml` para execução.
 - Arquivo `README.md` com instruções de instalação e execução.
 - Documento `documentacao.pdf` contendo a descrição arquitetural, justificativas de design e reflexões finais.
 - o Link do repositório deve ser enviado no moodle.

5. **Teste e reprodutibilidade.** O sistema entregue deve ser possível de executar em qualquer ambiente Linux com **Docker** e **docker-compose** instalados, sem necessidade de configuração adicional.
6. **Boas práticas de código e versionamento.** É recomendado que os grupos utilizem commits frequentes e mensagens claras no Git, além de seguir boas práticas de organização do código (separação por camadas, nomes descritivos de classes e funções, comentários quando necessário).

Assim, é esperado que a implementação demonstre explicitamente a **organização arquitetural** escolhida, na estrutura de código e organização necessária. Essa organização deve estar alinhada à justificativa apresentada na documentação e aos diagramas UML entregues.

2 Critérios de Avaliação

A avaliação do trabalho será baseada em quatro dimensões principais: **Modelagem UML**, **Arquitetura de Software**, **Implementação** e **Documentação**. A tabela abaixo apresenta os pesos de cada dimensão:

Critério	Peso
Modelagem UML	20%
Arquitetura de Software	30%
Implementação	30%
Documentação e Organização da Entrega	20%

Níveis de Desempenho

- **Modelagem UML (20%)**

- Excelente (17–20): Diagramas completos e consistentes, uso correto de padrões de projeto (se aplicável), clareza nos relacionamentos.
- Bom (11–16): Diagramas corretos mas com pequenas inconsistências ou incompletudes.
- Insuficiente (0–10): Diagramas ausentes, incorretos ou incoerentes com o domínio.

- **Arquitetura de Software (30%)**

- Excelente (21–30): Estilo arquitetural bem definido, refletido na organização do código e justificado na documentação.
- Bom (15–20): Estilo arquitetural definido mas com aplicação parcial ou justificativa incompleta.
- Insuficiente (0–14): Falta de clareza na escolha do estilo ou ausência de coerência entre modelo, código e documentação.

- **Implementação (30%)**

- Excelente (21–30): API REST completa (CRUD de espaços, reservas e pagamentos), executável via `docker compose up`, com documentação Swagger/OpenAPI, e Estilo arquitetural refletido na organização do código .
- Bom (16–20): API REST funcional mas com lacunas (ex.: ausência de um recurso ou documentação parcial), execução em Docker possível com ajustes mínimos. Estilo arquitetural incompleto na organização do código .
- Insuficiente (0–15): API incompleta, falha de execução em Docker, ausência de documentação mínima. Código que não reflete o estilo arquitetural escolhido.

- **Documentação e Organização (20%)**

- Excelente (11–20): Documento PDF claro e bem estruturado, README com instruções de execução, repositório Git organizado.
- Bom (6–10): Documentação existente mas incompleta ou pouco clara.
- Insuficiente (0–5): Ausência de documentação ou falta de instruções para execução.